

Juice Shop Threat Model

Owner: Salma Zrd
Reviewer: Project Supervisor
Contributors:
Date Generated: Fri Jul 10 2026

Executive Summary

High level system description

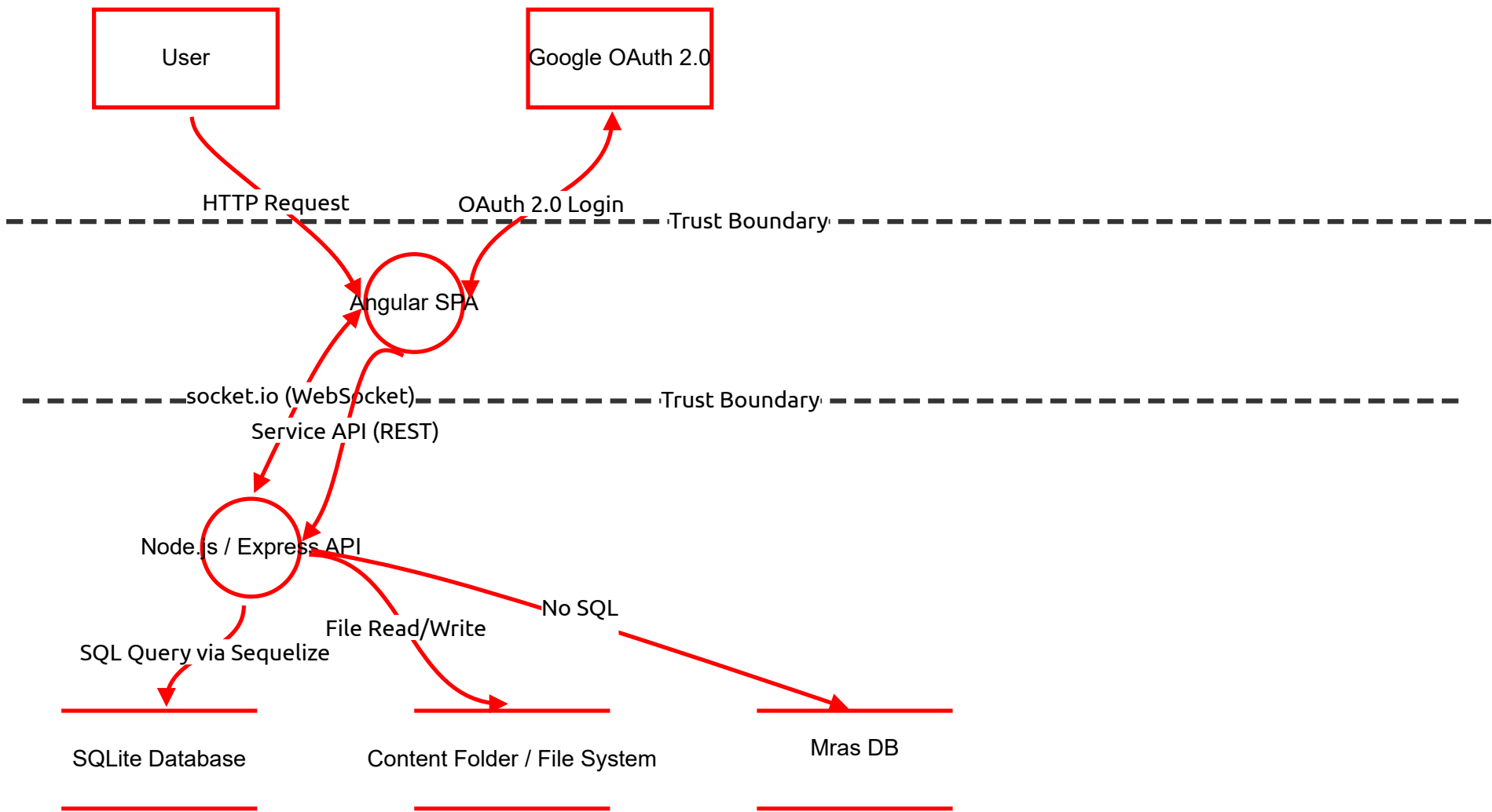
Threat model for OWASP Juice Shop developed as part of a DevSecOps internship project. The objective is to identify security threats using the STRIDE methodology before implementing automated security controls.

Summary

Total Threats	39
Total Mitigated	0
Total Open	39
Open / Critical Severity	8
Open / High Severity	18
Open / Medium Severity	11
Open / Low Severity	1
Open / TBD Severity	1

Juice Shop Architecture

High-level architecture of the OWASP Juice Shop application.



Juice Shop Architecture

User (Actor)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
32	Utilisateur malveillant — Brute Force login	Spoofing	High	Open		Un attaquant essaie des milliers de combinaisons email/mot de passe automatiquement pour accéder au compte d'un autre utilisateur. Juice Shop n'a pas de limite de tentatives.	Rate limiting sur le login, blocage après X tentatives échouées, CAPTCHA, authentification multi-facteur
33	Utilisateur malveillant — Attaque par replay (Replay Attack)	Spoofing	High	Open		Un attaquant intercepte un token JWT valide d'un autre utilisateur et l'utilise pour se faire passer pour lui dans ses requêtes.	Tokens JWT avec expiration courte, liste noire des tokens révoqués, lier le token à l'IP ou User-Agent
41	Répudiation d'actions	Repudiation	Medium	Open		Absence de traçabilité fiable côté serveur	l'utilisateur peut nier ses actions Journalisation côté serveur (qui, quoi, quand).

Google OAuth 2.0 (Actor)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
34	Phishing — Fausse page Google OAuth	Spoofing	High	Open		Un attaquant crée une fausse page de login Google qui ressemble à l'authentification OAuth légitime. L'utilisateur entre ses identifiants Google sur la fausse page — ils sont volés.	Afficher clairement l'URL de redirection, utiliser PKCE, éduquer les utilisateurs à vérifier l'URL
35	Token OAuth volé — Accès non autorisé	Spoofing	High	Open		Si le token OAuth retourné par Google est intercepté pendant la redirection, un attaquant peut l'utiliser pour se connecter à la place de l'utilisateur.	Utiliser PKCE (Proof Key for Code Exchange), valider le state parameter anti-CSRF, HTTPS obligatoire sur la redirect_uri

Angular SPA (Process)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
1	XSS — Vol de token JWT	Spoofing	High	Open		Un attaquant injecte du JavaScript malveillant dans un champ de recherche ou commentaire. Le script vole le token JWT stocké dans localStorage et l'envoie à un serveur distant.	Content Security Policy (CSP), encoder les sorties HTML, utiliser httpOnly cookies au lieu de localStorage
2	Manipulation des requêtes côté client	Tampering	High	Open		L'utilisateur modifie les paramètres d'une requête HTTP (prix, quantité, rôle) via les DevTools avant qu'elle parte vers l'API.	Toujours valider et vérifier côté serveur, ne jamais faire confiance aux données du client
3	Token JWT exposé dans localStorage	Information disclosure	Medium	Open		Le token d'authentification est stocké en clair dans le localStorage du navigateur, accessible par n'importe quel script JavaScript.	Stocker le token dans un cookie httpOnly inaccessible au JavaScript

Node.js / Express API (Process)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
4	SQL Injection sur le login	Spoofing	Critical	Open		Le formulaire de login est vulnérable à l'injection SQL. Payload : ' OR '1'='1' -- Permet de se connecter sans mot de passe et d'accéder à n'importe quel compte.	Utiliser des requêtes préparées (parameterized queries), ne jamais concaténer des données utilisateur dans le SQL
5	Modification non autorisée des données	Tampering	Critical	Open		L'API ne vérifie pas si l'utilisateur a le droit de modifier une ressource (commande, profil d'un autre). Un utilisateur peut modifier les données d'un autre.	Vérifier l'ownership de chaque ressource, implémenter une autorisation au niveau objet (BOLA)
6	Absence de logs d'audit	Repudiation	Medium	Open		Les actions sensibles (login, achat, modification) ne sont pas loggées avec suffisamment de détails. Un utilisateur malveillant peut nier ses actions.	Logger toutes les actions avec timestamp, IP, user ID, action effectuée
7	API expose des données sensibles	Information disclosure	High	Open		Les réponses de l'API retournent des champs sensibles non nécessaires (hash de mot de passe, données internes, stack traces en cas d'erreur).	Filtrer les réponses API, ne retourner que les champs nécessaires, désactiver les stack traces en production
8	Absence de rate limiting	Denial of service	High	Open		L'API n'a pas de limite sur le nombre de requêtes. Un attaquant peut envoyer des milliers de requêtes pour saturer le serveur (brute force, DDoS).	Implémenter un rate limiter (ex: express-rate-limit), bloquer les IPs après X tentatives échouées
9	Accès non autorisé à /api/admin	Elevation of privilege	Critical	Open		Les routes d'administration ne vérifient pas correctement le rôle de l'utilisateur. Un utilisateur normal peut accéder aux fonctions admin.	Vérifier le rôle côté serveur sur chaque route, ne pas se fier au rôle envoyé par le client
42	Point d'entrée d'injection NoSQL	Tampering	High	Open		L'API construit des requêtes MarsDB avec des données client non validées (détail au store MarsDB).	Validation de type, sanitisation (ex. express-mongo-sanitize)

SQLite Database (Store)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
10	Injection SQL directe	Tampering	Critical	Open		Les requêtes SQL sont construites par concaténation de chaînes. Un attaquant peut injecter du SQL pour lire, modifier ou supprimer toutes les données.	Requêtes préparées obligatoires, principe du moindre privilège sur la DB
11	Base de données non chiffrée	Information disclosure	High	Open		Le fichier SQLite est stocké en clair sur le disque. Si un attaquant accède au serveur, toutes les données sont immédiatement lisibles.	Chiffrer la base avec SQLCipher, chiffrer le disque au niveau OS
12	Pas de sauvegarde / backup	Denial of service	Medium	Open		Aucune stratégie de backup n'est définie. Une suppression accidentelle ou malveillante des données serait irréversible.	Sauvegardes automatiques régulières, politique de rétention des données
44	Absence de traçabilité des modifications	Repudiation	Low	Open		Modifications non journalisées.	Journalisation

Content Folder / File System (Store)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
16	Upload de fichier malveillant	Tampering	High	Open		L'application accepte des uploads de fichiers sans vérifier correctement le type réel du fichier. Un attaquant peut uploader un fichier .js ou .php déguisé en image.	Vérifier le MIME type réel (pas juste l'extension), scanner les fichiers uploadés, stocker hors du webroot
17	Path Traversal	Information disclosure	High	Open		Le nom de fichier fourni par l'utilisateur n'est pas sanitisé. Un attaquant peut utiliser "../..../etc/passwd" pour lire des fichiers système.	Sanitiser les noms de fichiers, utiliser des noms générés aléatoirement, jamais utiliser le nom fourni par l'utilisateur
18	Accès direct aux fichiers uploadés	Elevation of privilege	Medium	Open		Les fichiers uploadés sont accessibles directement via URL sans vérification d'authentification. N'importe qui peut accéder aux fichiers privés.	Servir les fichiers via l'API avec vérification d'authentification, pas directement depuis le disque
43	Écriture de fichiers arbitraires	Tampering	High	Open		Écriture avec un nom fourni par l'utilisateur -> écrasement/écriture de code.	Noms aléatoires, permissions restreintes sur uploads.

HTTP Request (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
20	Interception des données en transit (Man-in-the-Middle)	Information disclosure	High	Open		Si la communication se fait en HTTP (pas HTTPS), un attaquant sur le même réseau peut intercepter tout le trafic — mots de passe, tokens, données.	Forcer HTTPS partout, ajouter le header HSTS
22	Falsification des requêtes HTTP	Tampering	Medium	Open		Un attaquant peut intercepter et modifier les requêtes HTTP entre le navigateur et le serveur (si pas de HTTPS).	HTTPS obligatoire, validation côté serveur

SQL Query via Sequelize (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
27	Injection SQL via Sequelize mal utilisé	Tampering	Critical	Open		Si Sequelize est utilisé avec des requêtes raw() sans paramètres préparés, des injections SQL sont possibles même avec un ORM.	Utiliser uniquement les méthodes Sequelize avec paramètres (findOne, where: {}), éviter sequelize.query() avec concaténation

File Read/Write (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
29	Écriture de fichiers arbitraires	Tampering	High	Open		L'API écrit des fichiers sur le système en utilisant des noms fournis par l'utilisateur. Un attaquant peut écraser des fichiers système ou écrire du code exécutable.	Générer des noms de fichiers aléatoires, restreindre les permissions du dossier uploads
30	Path Traversal via nom de fichier	Information disclosure	High	Open		Le nom de fichier ../..../etc/passwd peut permettre de lire des fichiers en dehors du dossier autorisé.	Sanitiser le nom de fichier, utiliser path.basename() pour extraire seulement le nom, jamais le chemin complet

No SQL (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
39	Requête NoSQL construite avec des données non validées	Tampering	Critical	Open		L'API construit les requêtes MarsDB directement avec les données reçues du client sans validation. Code dangereux : db.find({ email: req.body.email }) Si req.body.email = {"\$gt": ""} → la requête devient : db.find({ email: {"\$gt": ""} }) → retourne TOUS les utilisateurs	Valider que req.body.email est bien une string, utiliser des schémas de validation (Joi, Yup), rejeter toute valeur qui n'est pas du bon type
40	New STRIDE threat	Information disclosure	TBD	Open		Provide a description for this threat	Provide remediation for this threat or a reason if status is N/A

OAuth 2.0 Login (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
31	Vol du code d'autorisation OAuth	Information disclosure	Medium	Open		Le code d'autorisation OAuth peut être intercepté dans les logs ou l'historique du navigateur si mal géré.	Utiliser PKCE (Proof Key for Code Exchange), code d'autorisation à usage unique

socket.io (WebSocket) (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
26	Données sensibles poussées via WebSocket	Information disclosure	Medium	Open		Les événements socket.io peuvent exposer des données sensibles à tous les clients connectés si le broadcast n'est pas bien ciblé.	Toujours cibler les événements socket.io à l'utilisateur concerné, pas en broadcast global

Service API (REST) (Data Flow)

Number	Title	Type	Severity	Status	Score	Description	Mitigations
25	Injection de données malveillantes via l'API	Tampering	Critical	Open		L'API reçoit des données sans les valider. Un attaquant envoie des payloads malveillants (SQL injection, XSS) directement via les requêtes API.	Valider et sanitiser toutes les entrées côté serveur
45	Altération en transit	Tampering	Medium	Open		Modification des requêtes entre navigateur et serveur si pas de HTTPS.	HTTPS + contrôle d'intégrité , validation côté serveur.
46	Interception en transit (MITM)	Information disclosure	High	Open		En HTTP (pas HTTPS), un attaquant sur le réseau intercepte mots de passe, tokens, données.	HTTPS partout
47	Déni de service sur le canal exposé	Denial of service	Medium	Open		Saturation du flux (REST).	Rate limiting, timeouts.

Mras DB (Store)

Description: Base de données NoSQL — Sessions, Challenges, Panier

Number	Title	Type	Severity	Status	Score	Description	Mitigations
36	Injection NoSQL — Manipulation des requêtes	Tampering	Critical	Open		Contrairement au SQL, l'injection NoSQL utilise des objets JavaScript malveillants. Exemple d'attaque sur un login : Au lieu de envoyer : <code>{ "email": "user@test.com", "password": "1234" }</code> L'attaquant envoie : <code>{ "email": {"\$gt": ""}, "password": {"\$gt": ""} }</code> MarsDB interprète <code>{"\$gt": ""}</code> comme "plus grand que vide" → toujours vrai → accès sans mot de passe !	Valider le type des entrées (string obligatoire), rejeter les objets JSON dans les champs qui attendent une string, utiliser un schéma de validation strict
37	Accès non autorisé aux données de session	Information disclosure	High	Open		MarsDB stocke les sessions utilisateurs. Si un attaquant accède à la base MarsDB, il peut voler toutes les sessions actives et se connecter en tant que n'importe quel utilisateur sans connaître leur mot de passe.	Chiffrer les données de session, stocker uniquement l'ID de session (pas les données), expiration automatique des sessions
38	Déni de service — Saturation de MarsDB	Denial of service	Medium	Open		MarsDB est une base légère sans mécanisme de protection contre les requêtes abusives. Un attaquant peut envoyer des milliers de requêtes simultanées pour saturer MarsDB et rendre l'application inaccessible.	Rate limiting sur les routes qui utilisent MarsDB, timeout sur les requêtes NoSQL, surveillance des performances